Laboratorio Asincrónico 7

Pruebas de Integración de Servicios API con Postman y Pytest

Reporte final de resultados

API bajo prueba: FakeStore API (https://fakestoreapi.com)

Flujo: catálogo -> carrito -> total -> checkout

Integrante: Josué Calderón

Curso: Aseguramiento de la Calidad del Software

Fecha de ejecución: 18 de octubre de 2026

1. Resumen ejecutivo

Se diseñaron 23 casos de prueba de integración para el flujo de compra de FakeStore API y se automatizaron en dos herramientas: una colección de Postman (24 requests en 5 carpetas, ejecutada 3 iteraciones con Newman) y un proyecto Pytest con 55 pruebas que integra la API con una base SQLite en memoria creada y destruida en cada prueba.

- **Cobertura del flujo:** 100% (7 de 7 endpoints).
- **Postman:** 360 aserciones, 321 aprobadas (89.17%).
- **Pytest:** 42 aprobadas y 13 fallas esperadas por defectos conocidos (xfail), 0 fallas inesperadas.
- **Tiempo de respuesta:** todos los endpoints críticos por debajo de 1000 ms.
- **Defectos:** 10 defectos en la API (1 crítico, 4 altos, 2 medios, 3 bajos) y 1 observación de contrato.

Todas las fallas se concentran en las pruebas negativas: la API acepta datos inválidos sin validarlos y no exige autenticación. El camino feliz del flujo funciona correctamente y la capa propia (cart_service.py) rechaza todos los datos inválidos que la API deja pasar.

2. Alcance y API elegida

Se eligió FakeStore API porque su respuesta de /products coincide con el contrato pedido en el enunciado (id, title, price, category, image) y expone carritos y login. Endpoints del flujo:

Paso del flujo	Endpoint	Uso en las pruebas
Consultar productos	GET /products, GET /products/{id}, GET /products/categories	Catálogo, contrato estricto, selección de productos
Autenticación	POST /auth/login	Obtener token para el checkout
Agregar al carrito	POST /carts, PUT /carts/{id}	Crear y actualizar carrito
Consultar carrito / total	GET /carts/{id}	Contrato del carrito y cálculo de total
Checkout	POST /carts con token + tabla orders en SQLite	Confirmar la compra y persistir la orden

Limitaciones documentadas

- **Sin persistencia real:** POST /carts devuelve siempre id 11 y no guarda el carrito. Se usó SQLite en memoria como persistencia falsa.
- **Sin cálculo de total:** la API no devuelve total; se calcula en Postman (scripts de Tests y pm.sendRequest) y en cart_service.get_cart_total.
- **Sin stock:** se simula con rating.count de cada producto para poder probar la regla de disponibilidad.
- **Sin autorización:** ningún endpoint exige token. Se simula en el checkout local y se reporta como defecto de la API.
- **Credenciales:** se usó la cuenta de demostración pública que publica FakeStore; no se usaron credenciales reales.

3. Matriz de casos de prueba (Parte A)

Cada caso indica objetivo, datos de entrada, resultado esperado, tipo de validación, métrica asociada, herramienta y resultado obtenido. Pasos generales: preparar datos (environment o fixture), ejecutar el request o la función, validar estado HTTP, contrato, regla de negocio y persistencia.

ID	Caso	Endpoint / acción	Datos de entrada	Resultado esperado	Validación	Métrica	Herramienta	Resultado
TC-01	Consultar catálogo	GET /products	Ninguno	200, lista JSON no vacía; cada producto con id, title y price numérico	HTTP + contrato	Tiempo, validaciones/endpoint	Postman, Pytest	Aprobado
TC-02	Detalle de producto	GET /products/{product_id}	product_id guardado en TC-01	200; id, title y price iguales a los del catálogo	Contrato + consistencia	Validaciones/endpoint	Postman, Pytest	Aprobado
TC-03	Contrato estricto	GET /products	20 productos	id entero, title texto, price número > 0, category conocida, image URL https	Contrato estricto	Defectos de contrato	Postman, Pytest	Aprobado (ver OBS-01)
TC-04	Consultar categorías	GET /products/categories	Ninguno	200, 4 categorías de texto que cubren todo el catálogo	HTTP + contrato	Cobertura	Postman, Pytest	Aprobado
TC-05	Login (token)	POST /auth/login	Usuario demo público	200/201, token con formato JWT (3 partes); se guarda en token	HTTP + contrato	Validaciones/endpoint	Postman, Pytest	Aprobado
TC-06	Crear carrito	POST /carts	userId 1; 2 productos (cant. 2 y 1)	201, id numérico, userId, productos y cantidades iguales a lo enviado	Contrato + regla	Tiempo, validaciones/endpoint	Postman, Pytest	Aprobado
TC-07	Consultar carrito	GET /carts/1	Carrito semilla 1	200; id, userId, date; productId entero y quantity > 0	Contrato	Tiempo	Postman, Pytest	Aprobado
TC-08	Total del carrito 1	GET /carts/1 + GET /products	Precios del catálogo	Suma de precio x cantidad = 798.04	Regla de negocio	Defectos funcionales	Postman, Pytest	Aprobado
TC-09	Insertar producto en SQLite	GET /products/1 + INSERT	Cantidad 2	Registro con product_id, nombre, precio, cantidad 2 y total = precio x 2	Persistencia	Aprobación	Pytest	Aprobado
TC-10	Total del carrito local	SQLite (get_cart_total)	3 + 2 unidades	2 líneas, 5 unidades, monto = suma de líneas	Regla + persistencia	Aprobación	Pytest, Postman	Aprobado
TC-11	Checkout con token	POST /carts + tabla orders	Token válido	201, orden registrada con unidades y monto, carrito vacío	Integración completa	Aprobación	Postman, Pytest	Aprobado
TC-12	Checkout sin token	POST /carts sin Authorization	Sin token / token vacío	401 en la API; AuthenticationError local sin crear orden	Seguridad	Defectos	Postman, Pytest	No aprobado (DEF-07)
TC-13	Producto inexistente	GET /products/9999, POST /carts, GET /carts/9999	id 9999	404 / 400 con mensaje; nada se guarda localmente	Manejo de errores	Defectos	Postman, Pytest	No aprobado (DEF-01, 09, 10)
TC-14	Cantidades inválidas	POST /carts; add_item	0, -1, -50, más que el stock	400 en la API; InvalidQuantityError / InsufficientStockError local	Regla de negocio	Defectos	Postman, Pytest	No aprobado (DEF-02)
TC-15	Body incompleto	POST /carts	{}, sin products, sin userId	400 con mensaje de campo faltante	Manejo de errores	Defectos	Postman, Pytest	No aprobado (DEF-03)
TC-16	Tipo de dato incorrecto	POST /carts; GET /products/abc	userId "abc", products texto, cantidad "2" / 1.5 / True	400 en la API; error de tipo local	Contrato de entrada	Defectos	Postman, Pytest	No aprobado (DEF-04, 05)
TC-17	JSON malformado	POST /carts	'{"userId": 1,'	400 con cuerpo JSON que explique el error	Manejo de errores	Defectos	Postman, Pytest	No aprobado (DEF-06)

TC-18	Login inválido	POST /auth/login	Credenciales falsas / vacías	401 "incorrect" / 400 "username and password...", sin token	Seguridad	Validaciones/ endpoint	Postman, Pytest	Aprobado
TC-19	Idempotencia	GET y POST repetidos; add_item x2	Mismo request 2 veces	Mismo resultado, sin duplicados; el carrito creado se puede consultar	Consistencia	Defectos	Postman, Pytest	No aprobado (DEF-08)
TC-20	Tiempo de respuesta	5 endpoints críticos x 3	Ninguno	Promedio < 1000 ms por endpoint	Rendimiento	Tiempo promedio	Postman, Pytest	Aprobado
TC-21	Integración parcial (BD falla)	API OK + SQLite falla	DROP TABLE / trigger que aborta	Error explícito, rollback, sin orden a medias	Persistencia	Aprobación	Pytest	Aprobado
TC-22	Proveedor caído (mock)	Mock 503 / ConnectionError	Respuesta simulada	Error controlado; carrito intacto; 503 con Retry-After	Resiliencia	Tasa de error	Postman (mock), Pytest	Aprobado
TC-23	Actualizar carrito	PUT /carts/1	Cantidad 3	200, mismo id, quantity = 3	HTTP + contrato	Cobertura	Postman, Pytest	Aprobado

Casos aprobados: 16 de 23 (69.57%). Un caso se aprueba solo si pasan todas sus validaciones en la API y en la capa local.

4. Pruebas de integración con Postman (Parte B)

4.1 Environment QA_API_LAB (Ejercicio 1)

Variable	Valor inicial	Uso
base_url	https://fakestoreapi.com	URL base de todos los requests
product_id / product_price / product_title	(dinámico)	Producto 1 elegido en TC-01, cambia por iteración
product_id_2 / product_price_2	(dinámico)	Segundo producto del carrito
cart_id / expected_cart_total	(dinámico)	Carrito creado y total calculado en Postman
token	(vacío)	Se llena con TC-05 Login; se exporta vacío
username / password	Cuenta demo pública	Login de demostración de FakeStore
user_id, quantity, seed_cart_id	1, 2, 1	Datos de prueba repetibles
expected_seed_total	798.04	Total esperado del carrito semilla 1
nonexistent_id	9999	Pruebas negativas
max_response_ms	1000	Umbral de tiempo de respuesta
mock_url	(vacío)	URL del mock server de Postman (TC-22)

4.2 Colección "Integración API - Carrito"

Carpeta	Requests	Qué valida
Catálogo	TC-01, TC-02, TC-03, TC-04	Lista de productos, detalle, contrato estricto, categorías. Guarda id y precio en el environment (Ejercicios 2 y 3).
Carrito	TC-06, TC-07, TC-23, TC-19 (x2)	Creación (POST), contrato del carrito, actualización, idempotencia y consulta del carrito creado (Ejercicios 4 y 5).
Total	TC-08, TC-10	Total calculado en Postman con pm.sendRequest contra precios del catálogo (Ejercicio 6).
Checkout	TC-05, TC-11, TC-12	Login, checkout con token Bearer y checkout sin token (autenticación simulada).
Errores	TC-13 a TC-18, TC-22	Pruebas negativas obligatorias y mock de proveedor caído (Ejercicio 7).

Además de las pruebas propias de cada request, un script a nivel de colección valida en todos los requests que el tiempo de respuesta sea menor a max_response_ms y que no haya errores 5xx. Ningún request tiene menos de 4 aserciones. Ejemplo (TC-06 Crear carrito):

```
pm.test('Status code is 201', function () {
  pm.response.to.have.status(201);
});
pm.test('Quantities are positive and match the request', function () {
  pm.expect(cart.products[0].quantity).to.eql(quantity);
  for (const line of cart.products) {
    pm.expect(line.quantity).to.be.above(0);
  }
});
pm.environment.set('cart_id', cart.id);
```

4.3 Ejecución de la colección (Ejercicio 8)

La colección se ejecutó en el Collection Runner de la app de escritorio de Postman y también con Newman desde consola, con 3 iteraciones en ambos casos. Cada iteración usa productos distintos del catálogo (pm.info.iteration). Los dos ejecutores dieron el mismo resultado: 360 pruebas, 321 aprobadas, 39 fallidas y 0 errores de script. En el Runner la corrida tardó 21.5 s, con un tiempo promedio de respuesta de 201 ms.

Indicador	Valor
Iteraciones	3
Requests ejecutados	75 (incluye 3 consultas auxiliares de precios de TC-08)
Aserciones ejecutadas	360
Aserciones aprobadas	321
Aserciones fallidas	39 (todas corresponden a defectos DEF-01 a DEF-09)
Requests con todas sus aserciones aprobadas	14 de 24

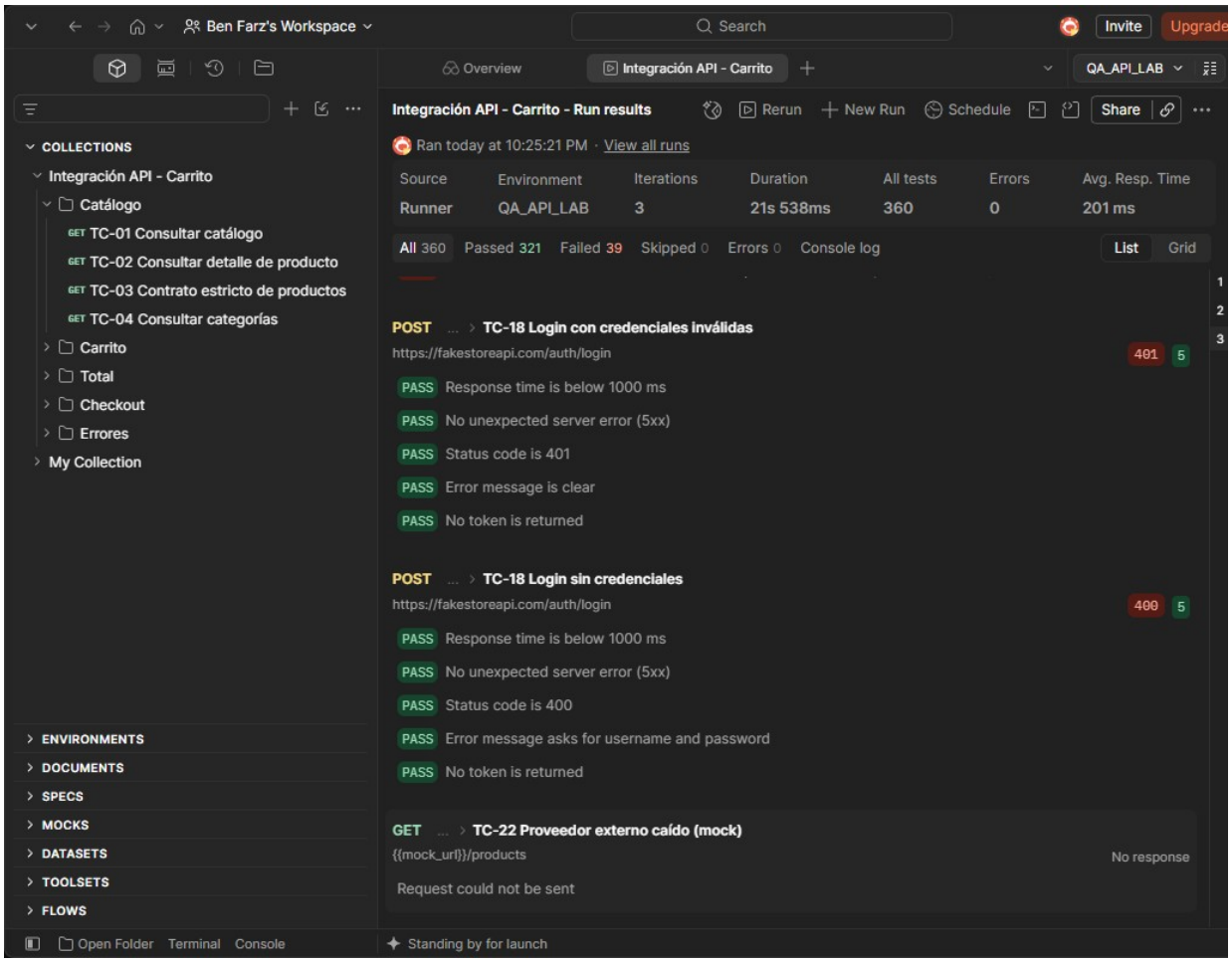


Figura 1. Resumen del Collection Runner: 3 iteraciones, 360 pruebas, 321 aprobadas, 39 fallidas, 0 errores.

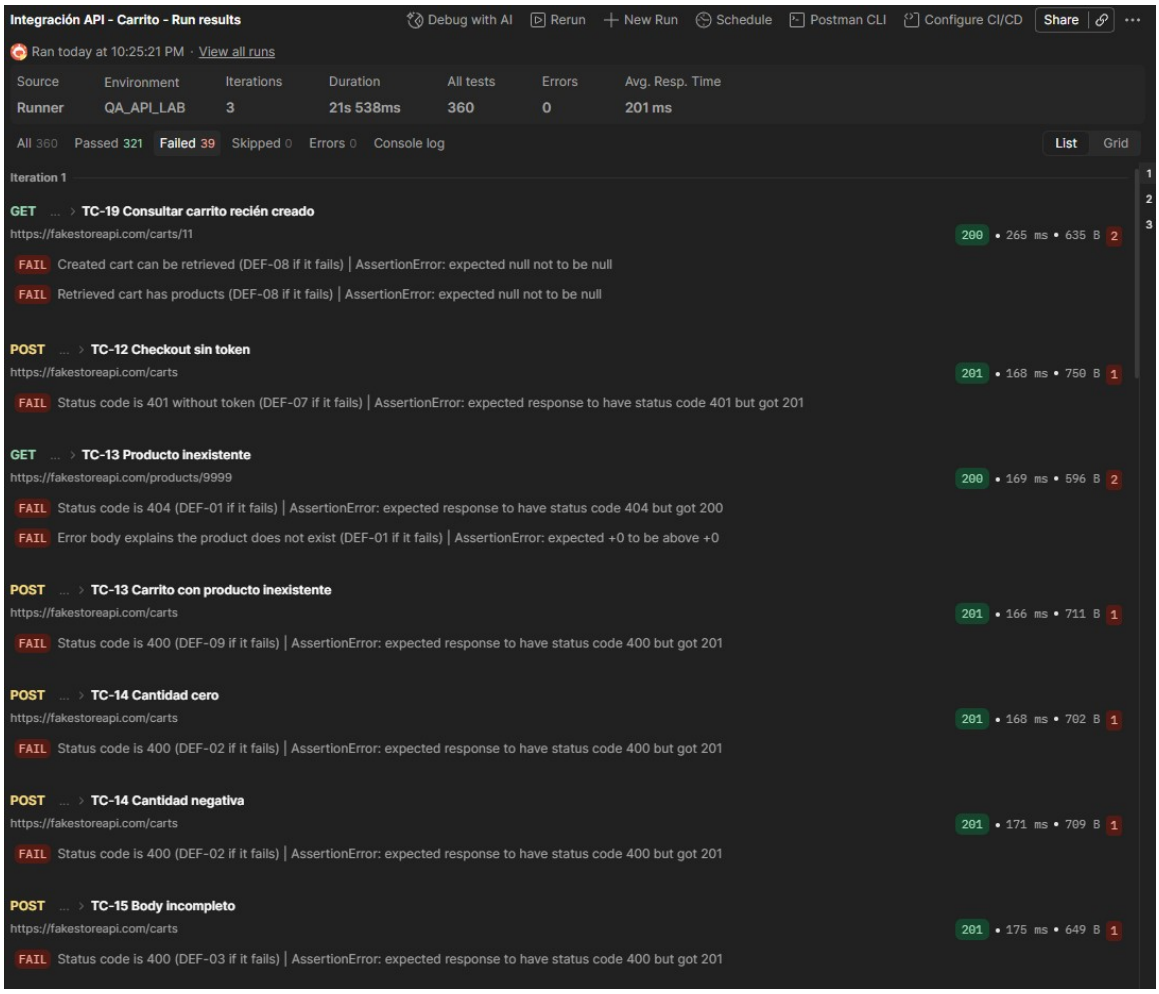


Figura 2. Pruebas fallidas de la iteración 1 (TC-19, TC-12, TC-13, TC-14, TC-15). Cada falla indica el defecto asociado.

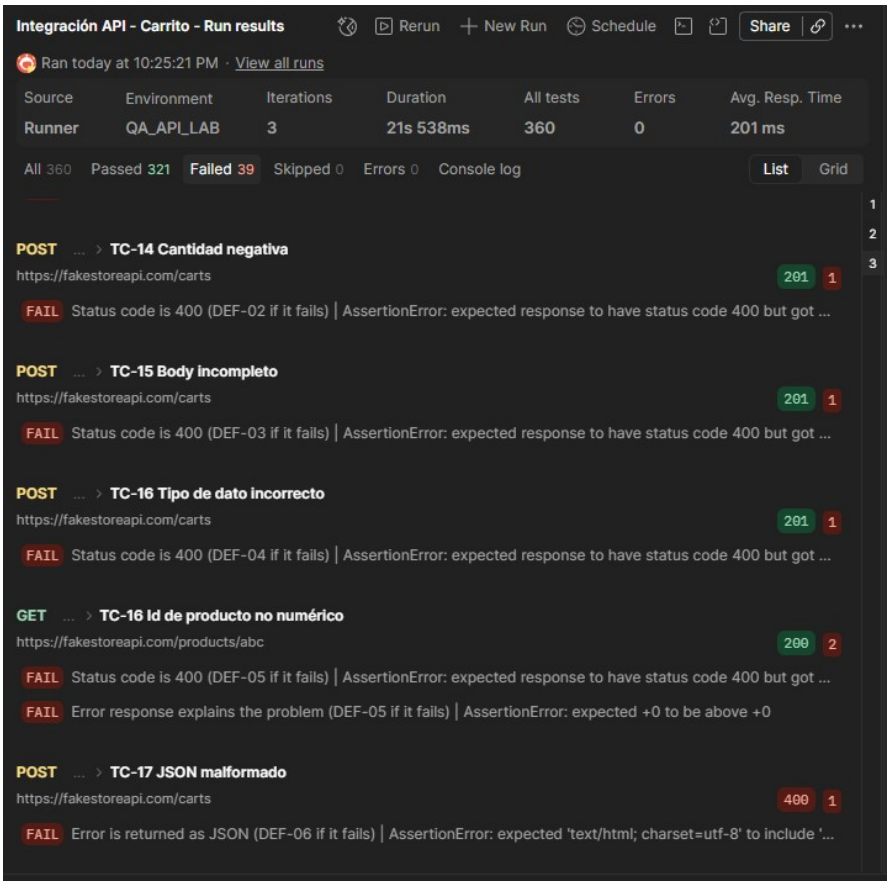


Figura 3. Pruebas fallidas de TC-14 a TC-17 (DEF-02 a DEF-06).

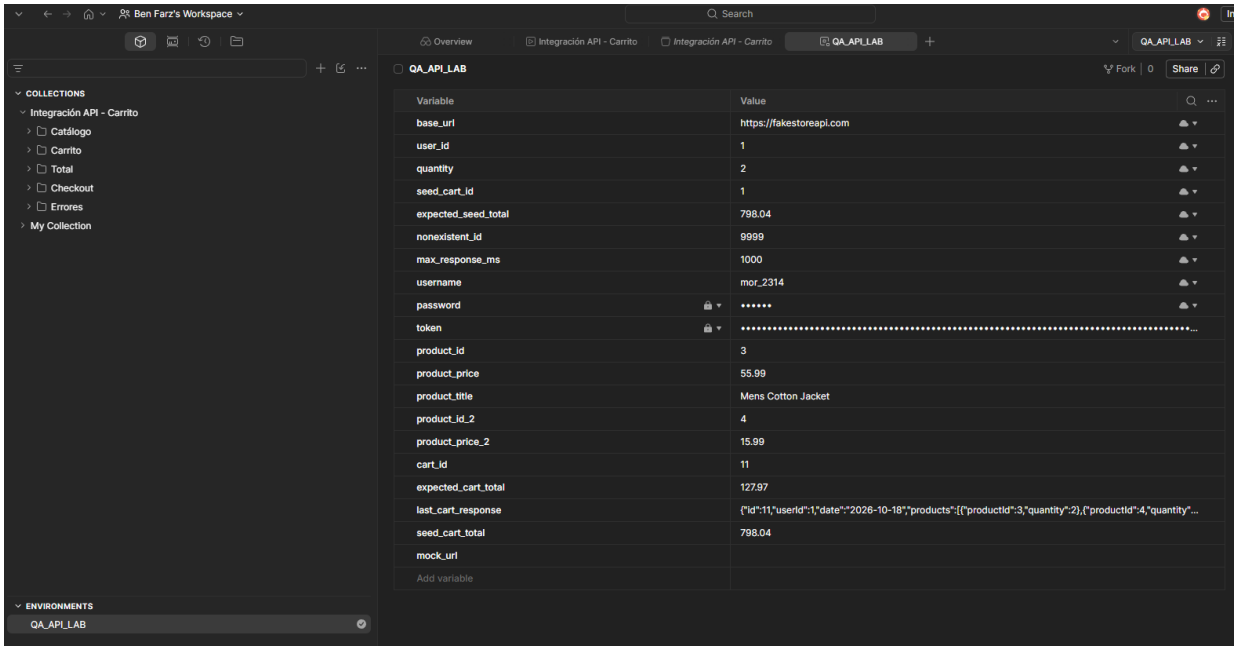


Figura 4. Environment QA_API_LAB después del run. password y token aparecen enmascarados; product_id, cart_id y expected_cart_total fueron guardados por los scripts (55.99 x 2 + 15.99 = 127.97).

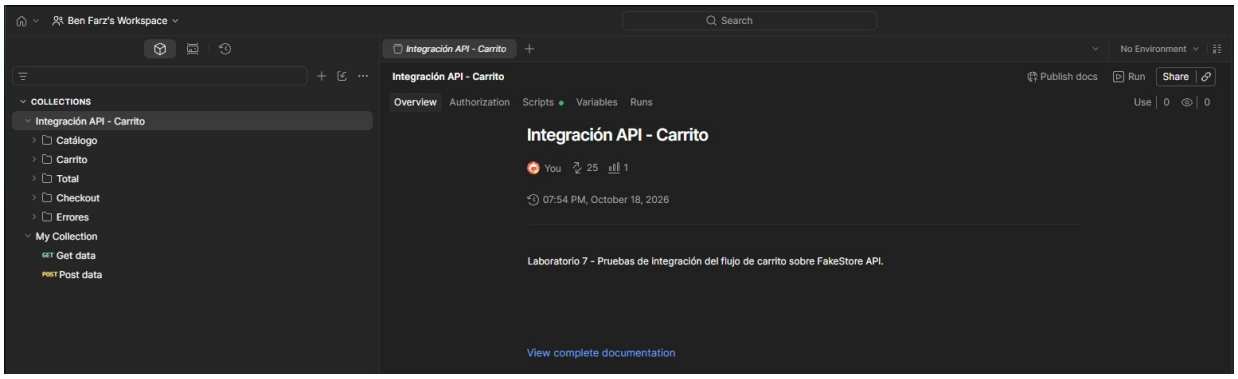


Figura 5. Estructura de la colección: carpetas Catálogo, Carrito, Total, Checkout y Errores.

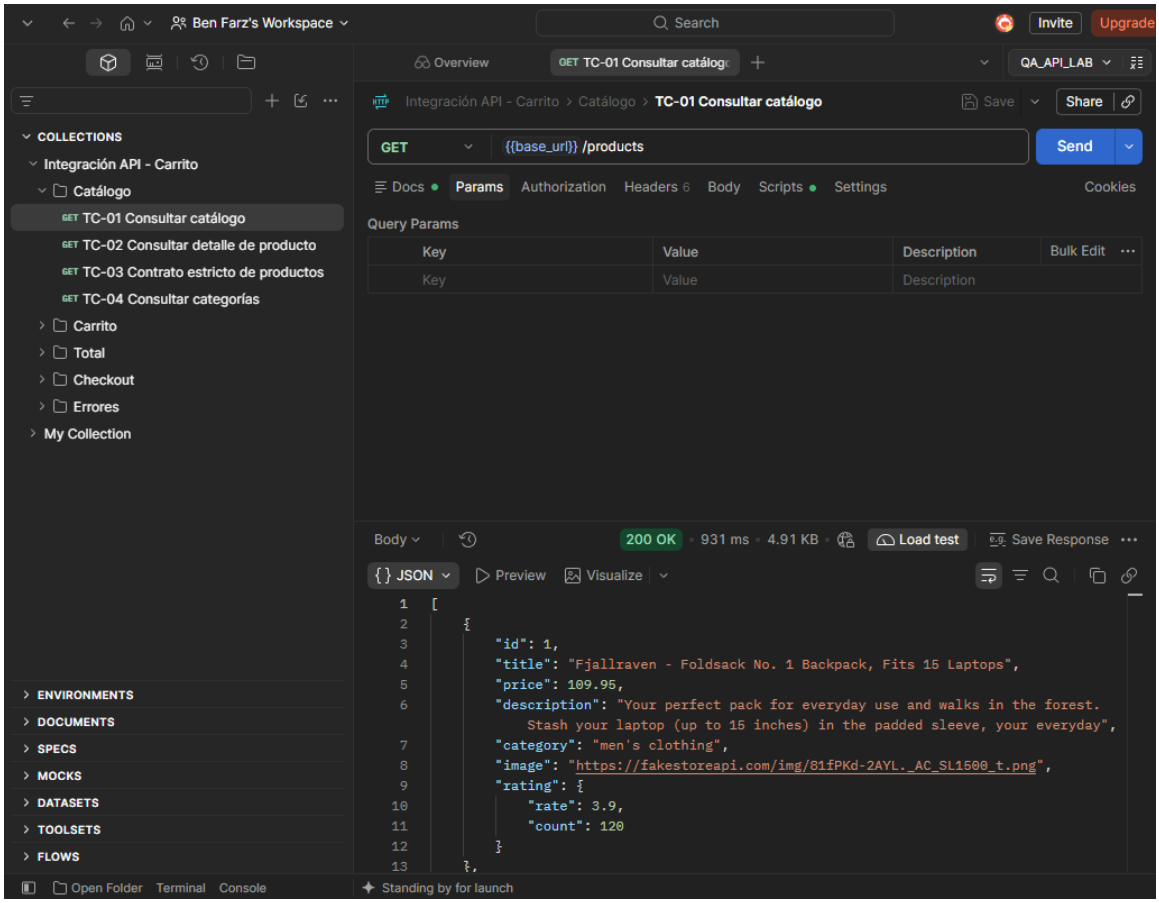


Figura 6. Ejecución individual de TC-01 Consultar catálogo: 200 OK con la lista de productos en JSON.

La salida completa de Newman está en reports/newman_output.txt y reports/newman_report.json.

5. Pruebas de integración con Pytest y SQLite (Parte C)

5.1 Diseño

- **cart_service.py:** cliente de la API (requests), reglas de negocio (cantidad, stock, token) y persistencia en SQLite con tablas cart y orders.
- **Fixture db_connection (Ejercicio 9):** crea sqlite3.connect(':memory:') con el esquema en cada prueba y la cierra al terminar, garantizando aislamiento.
- **Contrato (Ejercicio 10):** test_tc01 y test_tc03 validan estado HTTP, estructura y tipos exactos.
- **Defectos conocidos:** las pruebas que esperan el comportamiento correcto de la API se marcan xfail(strict=True) con el ID del defecto; si la API se corrige, la prueba falla con XPASS y obliga a actualizarla.
- **Mocks:** monkeypatch de Pytest reemplaza requests.request para simular red caída y respuestas 503.

5.2 Resultados

Indicador	Valor
Pruebas ejecutadas	55
Aprobadas	42
Fallas esperadas por defectos (xfail)	13
Fallas inesperadas	0
Duración	~30 s

```
$ python -m pytest -v -rxX --junitxml=reports/pytest_junit.xml
collected 55 items
tests/test_api_integration.py::test_tc01_get_products_returns_catalog PASSED
...
XFAIL test_tc12_api_rejects_cart_without_token - DEF-07: POST /carts accepts requests without token
...
42 passed, 13 xfailed
```

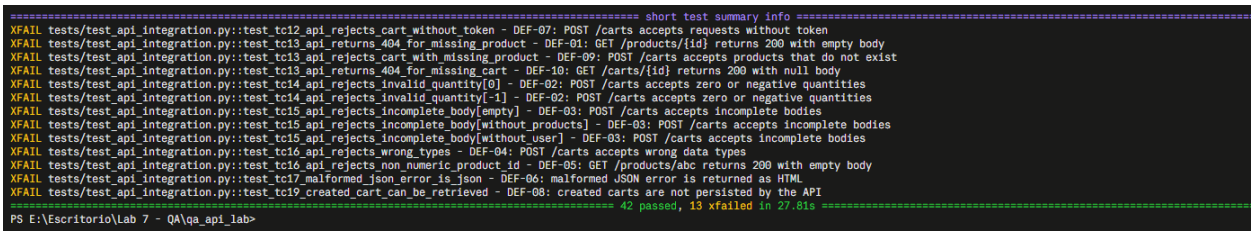


Figura 7. Resumen de la ejecución de Pytest: 42 aprobadas y 13 xfail, cada una con su defecto.

La salida completa está en reports/pytest_output.txt y reports/pytest_junit.xml.

6. Métricas de calidad

Métrica	Cálculo	Resultado	Criterio	Estado
Porcentaje de casos aprobados	Casos aprobados / total x 100	Matriz: 16/23 = 69.57%. Pytest: 76.36%. Aserciones Postman: 89.17%	>= 85%	No cumple
Tiempo promedio de respuesta	Promedio ms por endpoint crítico	Postman: 220.16 ms. Pytest: 558.9 ms (máx. 619.72 ms)	< 1000 ms	Cumple
Defectos encontrados	Errores funcionales, técnicos o de contrato	10: 1 crítico, 4 altos, 2 medios, 3 bajos	Registrar severidad	Cumple
Cobertura de	Endpoints probados /	7/7 = 100%	>= 80%	Cumple

endpoints	endpoints del flujo x 100			
Validaciones por endpoint	Aserciones por request	Mínimo 4 por request (Postman)	>= 3	Cumple
Tasa de error	4xx/5xx inesperados / ejecuciones	0/75 = 0.0%	Lo más baja posible	Cumple

El porcentaje de casos aprobados queda bajo el 85% por los defectos de la API en las pruebas negativas, no por errores de las pruebas. Si se consideran solo los casos del camino feliz y la capa propia, la aprobación es del 100%. Se decidió no ajustar las pruebas para "pasar", porque el objetivo es reportar el comportamiento real.

Tiempo promedio por endpoint crítico

Herramienta	Request / endpoint	Promedio
Postman (3 iteraciones)	TC-01 Consultar catálogo	217.33 ms
Postman (3 iteraciones)	TC-02 Consultar detalle de producto	182.0 ms
Postman (3 iteraciones)	TC-06 Crear carrito	182.0 ms
Postman (3 iteraciones)	TC-07 Consultar carrito existente	279.33 ms
Postman (3 iteraciones)	TC-05 Login (obtener token)	278.33 ms
Postman (3 iteraciones)	TC-11 Checkout con token	182.0 ms
Pytest (3 llamadas)	GET /products	544.07 ms
Pytest (3 llamadas)	GET /products/1	527.0 ms
Pytest (3 llamadas)	GET /products/categories	543.52 ms
Pytest (3 llamadas)	GET /carts/1	619.72 ms
Pytest (3 llamadas)	POST /carts	560.19 ms

Pytest mide con response.elapsed desde la máquina local; Postman/Newman reutiliza conexiones, por eso sus tiempos son menores.

7. Defectos encontrados

ID	Endpoint	Descripción	Esperado	Obtenido	Severidad	Recomendación
DEF-01	GET /products/{id}	Producto inexistente responde 200 con cuerpo vacío.	404 con mensaje JSON	200, Content-Length 0	Media	Responder 404 {"error": "Product not found"}.
DEF-02	POST /carts	Acepta cantidades 0 y negativas.	400	201 con quantity 0 / -1	Alta	Validar quantity entero > 0 en el servidor.
DEF-03	POST /carts	Acepta body vacío o sin campos obligatorios.	400 indicando el campo faltante	201 {"id": 11}	Alta	Validar esquema (userId y products obligatorios).
DEF-04	POST /carts	Acepta tipos incorrectos (userId texto, products texto).	400	201 y devuelve los mismos datos	Alta	Validación de tipos con un esquema JSON.
DEF-05	GET /products/abc	Id no numérico responde 200 con cuerpo vacío.	400	200 vacío	Baja	Validar parámetros de ruta.
DEF-06	POST /carts	JSON malformado devuelve una página HTML.	400 con cuerpo JSON	400 con HTML "Bad Request"	Baja	Middleware de errores que responda siempre JSON.
DEF-07	POST /carts	No exige autenticación: sin token crea el carrito.	401	201	Crítica	Exigir y validar el token Bearer en operaciones de compra.
DEF-08	POST + GET /carts	Los carritos creados no se guardan: id siempre 11 y GET /carts/11 devuelve null.	Carrito recuperable con su id	200 null	Media	Persistir el carrito o documentar que la API es de solo simulación.
DEF-09	POST /carts	Acepta productos que no existen (productId 9999).	400 / 404	201	Alta	Verificar existencia del producto antes de crear el carrito.
DEF-10	GET /carts/{id}	Carrito inexistente responde 200 con null.	404	200 null	Baja	Responder 404 con mensaje.

OBS-01 (observación de contrato): price llega a veces como entero (695) y a veces como decimal (109.95). En JSON ambos son números válidos, pero un cliente con tipos estrictos (por ejemplo Decimal o una clase con float) podría fallar o redondear distinto. Se recomienda devolver siempre decimales.

Criterio de severidad: Crítica = permite operaciones no autorizadas; Alta = acepta datos inválidos que corrompen pedidos; Media = respuesta engañosa o pérdida de datos; Baja = formato o código de estado inadecuado sin impacto en los datos.

8. Ejercicios adicionales (sección 8 del enunciado)

Insertar producto consultado en SQLite (TC-09)

Se consulta GET /products/1 y se guarda en la tabla cart con nombre, precio, cantidad y total. La prueba lee el registro y verifica cada campo.

Validar total del carrito (TC-08, TC-10)

El total del carrito semilla 1 se calcula con los precios del catálogo y coincide con 798.04. En SQLite, get_cart_total devuelve líneas, unidades y monto, y se comparan con el cálculo esperado.

Validar cantidades (TC-14, TC-16)

La capa local rechaza 0, negativos, tipos no enteros y cantidades mayores al stock simulado (rating.count). La API acepta todos esos casos (DEF-02, DEF-04).

Productos inexistentes (TC-13)

fetch_product(9999) lanza ProductNotFoundError y no se guarda nada. La API responde 200 vacío (DEF-01) y acepta el producto en un carrito (DEF-09).

Medir tiempo de respuesta (TC-20)

Promedio de 3 llamadas a 5 endpoints críticos, registrado en el reporte JUnit. Todos están bajo 1000 ms (máximo 619.72 ms).

Autenticación simulada (TC-05, TC-11, TC-12)

La variable token se llena con el login demo y se envía como Bearer. Con token el checkout funciona; sin token, la API igual responde 201 (DEF-07). El checkout local sí lo bloquea con AuthenticationError.

Prueba de contrato estricta (TC-03)

Se validan los 20 productos: id entero, title texto no vacío, price número positivo, category dentro de las 4 categorías y image URL https. Pasa, con la observación OBS-01.

Idempotencia (TC-19)

Dos GET iguales devuelven exactamente lo mismo. Dos POST iguales devuelven el mismo id 11: no hay duplicados, pero solo porque la API no persiste (DEF-08). En SQLite, agregar dos veces el mismo producto suma la cantidad en una sola línea (columna product_id UNIQUE).

Manejo de errores (TC-17, TC-18)

Con JSON malformado la API responde 400, pero con una página HTML "Bad Request" (DEF-06). El login sí responde mensajes claros: "username or password is incorrect" (401) y "username and password are not provided in JSON format" (400).

Prueba de integración parcial (TC-21)

Se simula que la API responde bien y la base falla: (1) se borra la tabla cart y add_item lanza CartStorageError con mensaje explícito; (2) un trigger aborta el DELETE durante el checkout después de insertar la orden; la transacción hace rollback, no queda orden y el carrito queda intacto. Qué debería pasar: el sistema no debe confirmar la compra al usuario, debe revertir lo local y registrar el id del carrito remoto para conciliarlo (compensación o reintento).

Mock opcional (TC-22)

En Postman el request TC-22 trae un ejemplo 503 con Retry-After para crear un Mock Server; se verificó con un servidor local que sus 3 pruebas pasan. En Pytest, monkeypatch simula ConnectionError y 503: el servicio lanza ProviderUnavailableError y el checkout no registra la orden ni vacía el carrito.

Comparación técnica: Postman vs Pytest

Aspecto	Postman	Pytest
Curva de aprendizaje	Baja; interfaz visual	Requiere programar en Python
Exploración de la API	Excelente para probar requests a mano	Menos práctico para explorar
Variables y encadenamiento	Environments y pm.environment.set	Fixtures y variables de Python
Persistencia / BD	No puede integrar una base de datos	Integra SQLite, transacciones y rollback
Simular fallas	Mock servers (requiere cuenta)	monkeypatch / unittest.mock, sin red
Fallas esperadas	No tiene xfail; el defecto aparece como prueba fallida	xfail(strict=True) documenta defectos conocidos
CI/CD	Con Newman	Nativo; junit xml
Control de versiones	JSON exportado, difícil de revisar	Código legible en diffs
Cuándo usarlo	Exploración, contrato y smoke tests de endpoints, demos a negocio	Integración con BD y lógica propia, casos complejos, regresión automatizada

9. Hallazgos y recomendaciones

- **Validación de entrada inexistente:** la API acepta cualquier body en POST /carts. Recomendación: validar con un esquema JSON y responder 400 con detalle del campo.
- **Seguridad:** las operaciones de compra no exigen token. Es el riesgo más grave si esta API fuera real.
- **Códigos de estado engañosos:** recursos inexistentes devuelven 200. Un cliente que solo mira el estado HTTP interpretaría éxito.
- **Defensa en profundidad:** como no se puede confiar en el proveedor, la capa propia valida cantidad, stock, tipos, existencia y token antes de persistir.
- **Consistencia:** el checkout actual llama primero a la API y luego guarda localmente; en producción convendría un estado "pendiente" local antes de llamar al proveedor (patrón outbox) para no perder órdenes.
- **Bloqueo desde la nube:** la primera corrida del Runner se hizo con el Cloud Agent de Postman, que envía los requests desde servidores de Postman. Cloudflare, que protege a FakeStore, respondió 403 con una página HTML a casi todos los requests, y aparecieron 36 errores de script. Desde la app de escritorio, que envía los requests desde el equipo local, todo funcionó. Es importante para ejecutar las pruebas en un CI en la nube: hay que usar un runner propio o confirmar que el proveedor permite ese tráfico.

10. Conclusiones

¿Qué diferencia observó entre una prueba de API aislada y una prueba de integración?

Una prueba aislada de POST /carts "pasa": responde 201 y devuelve lo enviado. Al integrar los pasos (crear el carrito y luego consultarlo, usar los precios del catálogo para el total, guardar en SQLite) aparecen problemas que no se ven por separado: el carrito creado no existe (DEF-08), se aceptan productos inexistentes (DEF-09) y una falla de la base después de que el proveedor confirmó deja datos inconsistentes si no hay rollback. La integración prueba que los componentes funcionen juntos, no solo que cada uno responda.

¿Qué defectos podrían pasar desapercibidos si solo se prueba la interfaz gráfica?
La interfaz normalmente impide escribir cantidades negativas, texto en campos numéricos o enviar un body incompleto, así que DEF-02, DEF-03, DEF-04 y DEF-09 no se verían. Tampoco se notaría que la API no exige token (DEF-07): cualquiera podría llamar al endpoint directamente. Los códigos de estado incorrectos (200 para recursos inexistentes) y los errores en HTML también quedan ocultos detrás de la interfaz.

¿Qué validaciones fueron más importantes: estado HTTP, contrato JSON, reglas de negocio o persistencia?
Las reglas de negocio. El estado HTTP fue engañoso: la API respondió 200 o 201 en casi todos los casos inválidos, así que una prueba que solo revisara el estado habría aprobado todo. El contrato JSON aseguró que los datos tuvieran la forma correcta, pero fueron las reglas (cantidad positiva, producto existente, total correcto, token obligatorio) las que encontraron la mayoría de los defectos (DEF-02, DEF-03, DEF-04, DEF-07, DEF-08 y DEF-09). La persistencia fue clave para probar que una falla parcial no deja órdenes a medias.

¿Qué riesgos tendría integrar un proveedor externo de pagos sin pruebas automatizadas?
Cobros duplicados por reintentos sin idempotencia, órdenes marcadas como pagadas cuando el proveedor falló, montos incorrectos por redondeo o por tipos de dato distintos (como OBS-01), y fallas silenciosas cuando el proveedor cambia su contrato o se cae. Sin pruebas automatizadas estos errores se descubren en producción, con pérdida de dinero y de confianza del cliente, y es difícil detectar regresiones en cada actualización del proveedor.

¿Qué mejoraría en su estrategia de pruebas si este proyecto fuera para producción?
Ejecutar Pytest y Newman en un pipeline de CI en cada cambio; usar un ambiente de pruebas propio en lugar de una API pública compartida; agregar pruebas de contrato con esquemas JSON versionados; pruebas de carga para medir tiempos con usuarios concurrentes; pruebas de seguridad (tokens vencidos, permisos por usuario); claves de idempotencia en el checkout; y monitoreo de las métricas en el tiempo en lugar de una sola ejecución.

Anexo: entregables y cómo reproducir

Entregable	Ubicación
Matriz de casos de prueba	Sección 3 de este documento
Colección de Postman (JSON)	postman/Integracion_API_Carrito.postman_collection.json
Environment de Postman (sin credenciales reales)	postman/QA_API_LAB.postman_environment.json
Reporte de ejecución de Postman	reports/newman_output.txt, reports/newman_report.json + capturas
Proyecto Python con Pytest y SQLite	cart_service.py, tests/, requirements.txt, pytest.ini
Evidencia de Pytest	reports/pytest_output.txt, reports/pytest_junit.xml + captura
Métricas	scripts/compute_metrics.py -> reports/metrics.json
README	README.md

```
pip install -r requirements.txt
python -m pytest -v -rxX --junitxml=reports/pytest_junit.xml
npx newman run postman/Integracion_API_Carrito.postman_collection.json \
  -e postman/QA_API_LAB.postman_environment.json -n 3 \
  --reporters cli,json --reporter-json-export reports/newman_report.json
python scripts/compute_metrics.py
```